

Redes de Comunicaciones: NanoFiles



Nombre: Juan Hernández López. **Correo:** juan.h.l@um.es.

DNI: 49196501-F

Curso 2º. Grupo: 3.4

Asignatura: Redes de Comunicaciones

Profesor: Juan J. Pujante Moreno
Facultad de Informática. Universidad de Murcia

Fecha de entrega: 15/3/2026

ÍNDICE

1.- Introducción.....	3
2.- Formato de los mensajes del protocolo de comunicación con el Directorio.....	3
2.1.- Tipos y descripción de los mensajes.....	3
2.1.1.- Ping.....	3
2.1.2.- Dirfiles.....	4
2.1.3.- Peers.....	4
2.1.4.- Serve.....	4
3.- Formato de los mensajes del protocolo de transferencia de ficheros.....	5
3.1.- Tipos y descripción de los mensajes.....	5
3.1.1.- FileListRequest.....	5
3.1.2.- FileListResponse.....	5
3.1.3.- GetFile.....	5
3.1.4.- FileData.....	6
3.1.5.- FileNotFound.....	6
4.- Autómata de protocolo.....	6
4.1.- Autómata rol cliente de directorio.....	7
4.2.- Autómata rol servidor de directorio.....	8
4.3.- Autómata rol cliente de ficheros.....	8
4.4.- Autómata rol servidor de ficheros.....	9
5.- Ejemplo de intercambio de mensajes.....	10

1.- Introducción

En este documento se especifica el diseño de los protocolos de comunicación del programa NanoFiles, un sistema P2P para compartir ficheros que utiliza un servidor de directorio y múltiples peers que actúan como cliente y/o servidor

Se han diseñado dos protocolos: el primero, entre peers y directorio, sobre UDP con mensajes en formato “campo:valor”, y el segundo, entre peers, sobre TCP con mensajes binarios para listar y transferir ficheros.

2.- Formato de los mensajes del protocolo de comunicación con el Directorio

Para definir el protocolo de comunicación con el Directorio, vamos a utilizar mensajes de texto con formato “campo:valor”. El valor que tome el campo operation (código de operación) indicará el tipo de mensaje y por tanto su formato.

Cada mensaje está compuesto por una o más líneas, donde cada línea contiene un campo seguido de ‘:’ y su valor. Las líneas terminan con \n (retorno de carro) y el mensaje completo termina con una línea vacía (\n\n), y este formato permite mensajes de longitud variable.

2.1.- Tipos y descripción de los mensajes

2.1.1.- Ping

- Mensaje: **ping**
- Sentido de la comunicación; **Cliente** —> **Directorio**
- Descripción: Este mensaje lo envía el cliente al Directorio para comprobar que está activo y utiliza un protocolo compatible (comando ping). Una vez enviado el ping, el directorio responde confirmando la compatibilidad o el rechazo hacia el ping.
- Cliente envía:
 - operation: ping
 - protocol. 123456789A
- Directorio responde:
 - operation: ping
 - protocol: 123456789A / 000000000X (compatible / no compatible)

2.1.2.- Dirfiles

- Mensaje: **dirfiles**
- Sentido de la comunicación: **Cliente** —> **Directorio**
- Descripción: El cliente solicita la lista de ficheros del directorio. El directorio responde con todos los ficheros de su carpeta dir-shared.
- Cliente solicita:
 - operation: dirfiles
 - protocol: 123456789A
- Directorio responde enviando:
 - operation: dirfiles
 - protocol: 123456789A
 - files:

prueba1.txt	40	864bc45cb5b18683508c40eb865684fd57f7afdd
prueba3.txt	140	256fde20842e608954a1187d4b270cd04c1bc438
prueba2.txt	131	b9b3841dd57e08f1417a9160760c5610d33bf569

2.1.3.- Peers

- Mensaje: **peers**
- Sentido de la comunicación: **Cliente** —> **Directorio**
- Descripción: El cliente solicita al directorio el número de peers registrados como servidores de ficheros. El directorio responde con la lista de peers con el nickname y la dirección IP con el puerto. Si no hay ninguno, devuelve la lista vacía.
- Cliente solicita:
 - operation: peers
 - protocol: 123456789A
- Directorio responde enviando (si no hay peers registrados):
 - (none)
- Directorio responde enviando (si hay peers registrados):
 - operation: peers
 - protocol: 123456789A
 - peers: lou4 127.0.0.1 9999

2.1.4.- Serve

- Mensaje: **serve**
- Sentido de la comunicación: **Cliente** —> **Directorio**
- Descripción: Cuando se ejecuta el comando serve, el peer arranca su servidor de ficheros en un puerto libre, y a continuación, envía un mensaje al directorio para registrarse como servidor

3.- Formato de los mensajes del protocolo de transferencia de ficheros

Para definir el protocolo de comunicación de transferencia de ficheros, vamos a utilizar mensajes binarios. El valor que tome el campo opcode (1 byte) indicará el tipo de mensaje y por tanto cuál es su formato, es decir, qué campos vienen a continuación. La clase PeerMessage modela estos mensajes, con métodos para crear cada tipo y métodos para leer o escribir los mensajes.

3.1.- Tipos y descripción de los mensajes

3.1.1.- FileListRequest

- Mensaje: **FileListRequest (opcode = 1)**
- Sentido de la comunicación: **Cliente** —> **Servidor**
- Descripción: Este mensaje lo envía el cliente al ejecutar el comando perfiles para solicitar la lista completa de ficheros disponibles en el servidor de ficheros. No contiene campos adicionales, solo el opcode indica la petición del listado.
- Ejemplo:

Opcode = 1

3.1.2.- FileListResponse

- Mensaje: **FileListResponse (opcode = 2)**
- Sentido de la comunicación: **Servidor** —> **Cliente**
- Descripción: Es una respuesta al FileListRequest. El servidor envía el número de ficheros disponibles seguido de la información de cada uno (nº, nombre, tamaño, hash) para que, cuando le llegue al cliente, éste imprima dicha información.
- Ejemplo:

Opcode = 2
numFiles: 1
fileName (UTF): prueba23.txt
fileSize (long): 38
fileHash (UTF): a99c506868eb85bbff45e419fb3347ed30344e7c

3.1.3.- GetFile

- Mensaje: **GetFile (opcode = 3)**
- Sentido de la comunicación: **Cliente** —> **Servidor**
- Descripción: Este mensaje se envía tras obtener la lista con perfiles y seleccionar un fichero por subcadena de hash. El servidor buscará ficheros cuyo hash contenga exactamente dicha subcadena.
- Ejemplo:

Opcode = 3
subcadenaFileHash (UTF): a99c50

3.1.4.- FileData

- Mensaje: **FileData (opcode = 4)**
- Sentido de la comunicación: **Servidor —> Cliente**
- Descripción: El servidor envía los datos del fichero (nombre, tamaño, hash, longitud_datos, data[]) (bytes del fichero completo) o fragmento tras una petición “peerdl <nickname>”. El cliente guarda el fichero con el nombre y verifica que el hash recibido coincide con el hash que se calcula.
- Ejemplo:

```
Opcode = 4
fileName (UTF): prueba23.txt
fileSize (long): 38
fileHash (UTF): a99c506868eb85bbff45e419fb3347ed30344e7c
dataLength (int): 38
data[38 bytes]: contenido binario completo del fichero
```

3.1.5.- FileNotFound

- Mensaje: **FileNotFound (opcode = 5)**
- Sentido de la comunicación: **Servidor —> Cliente**
- Descripción: El servidor responde cuando no encuentra fichero con el hash (subcadena del hash no válido) y una vez comprobado, el cliente imprime un mensaje de error (mensajeError).
- Ejemplo:

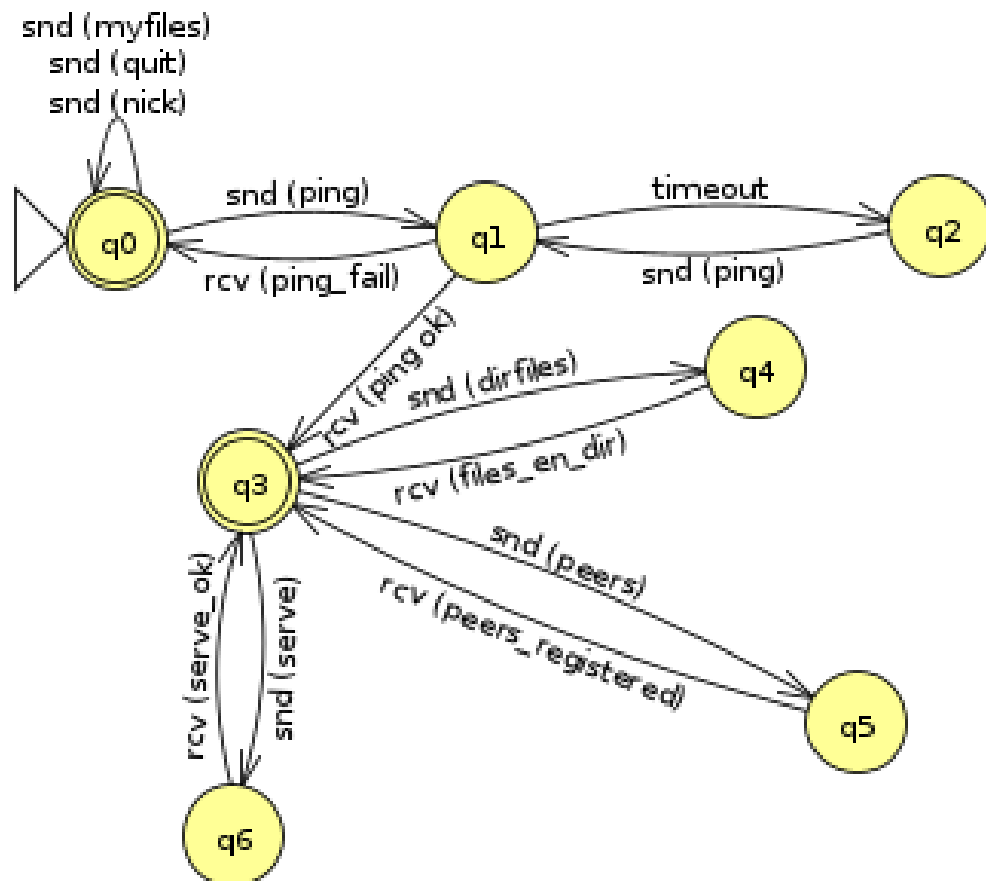
```
Opcode = 5
mensajeError (UTF): “Fichero no encontrado”
```

4.- Autómata de protocolo

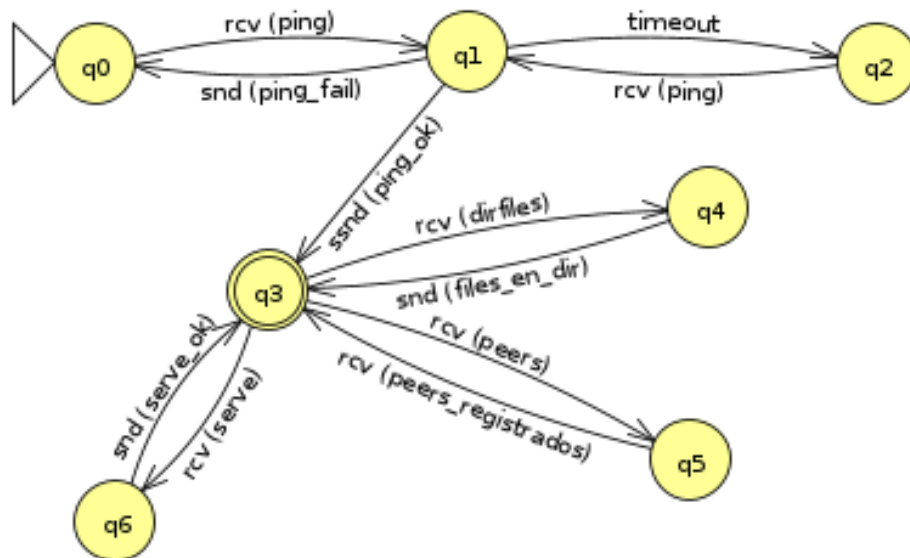
A continuación vamos a detallar los diferentes autómatas dependiendo del rol que queramos plasmar, ya sea como cliente o servidor, y si nos comunicamos con el directorio o con el servidor de ficheros.

- La restricción que tiene el autómata es que primero se debe realizar un ping antes de ejecutar cualquier comando de comunicación con el directorio, además, dicho ping tiene que ser válido, es decir, que el ping se haya recibido correctamente.

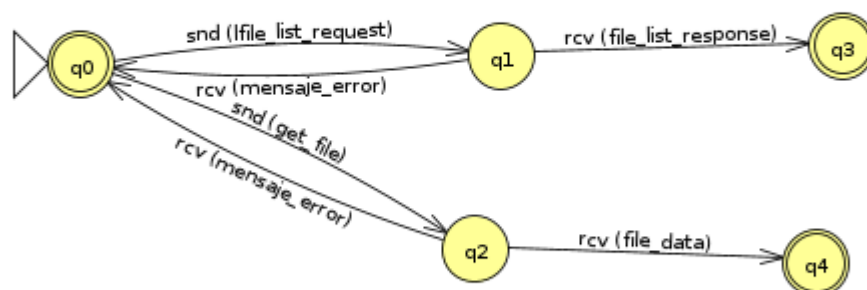
4.1.- Autómata rol cliente de directorio



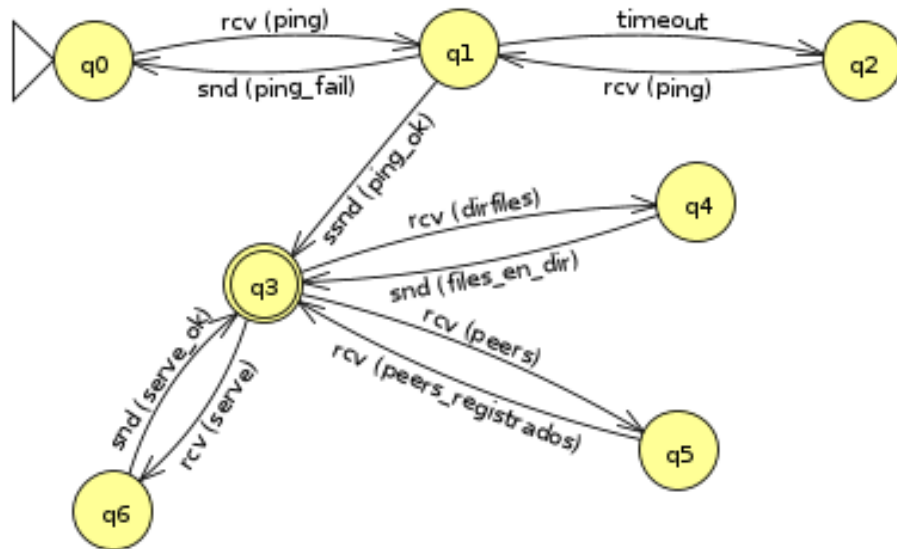
4.2.- Autómata rol servidor de directorio



4.3.- Autómata rol cliente de ficheros



4.4.- Autómata rol servidor de ficheros



5.- Ejemplo de intercambio de mensajes

Aquí en esta sección vamos a realizar un ejemplo de intercambios de mensajes. Lo que primero vamos a hacer es probar nuestros primeros estados del autómata de comunicación con el directorio donde podremos comprobar que no es necesario hacer un ping para realizar los siguientes comandos (estos comandos no se comunican con el directorio)

```
* Peer nickname: eva0
NanoFiles shell
For help, type 'help'
Do you want to use 'localhost' as location of the directory server? (y/n): y
Using directory location: localhost
(nanoFiles@nf-shared) nick juan
* Nickname changed to: juan
(nanoFiles@nf-shared) myfiles
List of files in local folder:
Name                               Size Hash
prueba23.txt                       38
a99c506868eb85bbff45e419fb3347ed30344e7c
```

Ahora, vamos a comprobar nuestra restricción de que primero tenemos que realizar un ping para realizar cualquier tipo de comando de comunicación con el directorio:

```
(nanoFiles@nf-shared) dirfiles
* You must ping the directory before using this command
```

Una vez hecho esto, lo que vamos a proceder a hacer es un ping para ver que hay compatibilidad con el directorio:

```
(nanoFiles@nf-shared) ping
* Checking if the directory at localhost is available...
Intento 1: Datagrama enviado
Respuesta recibida correctamente.
Directorio alcanzable y compatible (boletín ASCII)
* Directory is active and uses compatible protocol 123456789A
```

```
Directory received datagram from /127.0.0.1:36318 of size 36 bytes
Data received (boletín ASCII): operation:ping
protocol:123456789A
Ping compatible cuyo protocolo tiene un ID: 123456789A
Enviando respuesta (boletín ASCII): operation:ping
protocol:123456789A
```

Cuando se ha realizado el ping, entonces vamos a seguir con un ejemplo de ejecución de los diferentes comandos:

```
(nanoFiles@nf-shared) dirfiles
Intento 1: Datagrama enviado
Respuesta recibida correctamente.
Ficheros que se reciben del directorio (3)
* These are the files tracked by the directory at localhost
Name                               Size Hash
prueba1.txt                        40
864bc45cb5b18683508c40eb865684fd57f7afdd
prueba3.txt                        140
256fde20842e608954a1187d4b270cd04c1bc438
prueba2.txt                        131
b9b3841dd57e08f1417a9160760c5610d33bf569
```

```
Directory received datagram from /127.0.0.1:36318 of size 40 bytes.
Data received (boletín ASCII): operation:dirfiles
protocol:123456789A
Procesando dirfiles...
Enviando respuesta (boletín ASCII): operation:dirfiles
protocol:123456789A
files:prueba1.txt                  40
864bc45cb5b18683508c40eb865684fd57f7afdd
prueba3.txt                        140
256fde20842e608954a1187d4b270cd04c1bc438
prueba2.txt                        131
b9b3841dd57e08f1417a9160760c5610d33bf569
```

```
(nanoFiles@nf-shared) peers
* Registered peers at localhost
Intento 1: Datagrama enviado
Respuesta recibida correctamente.
(none)
```

```
Directory received datagram from /127.0.0.1:36318 of size 37 bytes.
Data received (boletín ASCII): operation:peers
protocol:123456789A
Procesando peers...
Enviando respuesta (boletín ASCII): operation:peers
protocol:123456789A
```